```
void insert(OrderId key, Order* value) {
    size_t idx = hash(key);
    while (table_[idx].is_occupied) {
        if (table_[idx].key == key) {
            table_[idx].value = value; // Update existing
            return;
        }
        idx = (idx + 1) % TableSize; // Linear probe
    }
    table_[idx] = {key, value, true};
}
```

initial target

collision checks

check if we are updating an existing key

check the next slot

Once an empty slot is found we write our key-value pair

```
Order* find(OrderId key) const {
    size_t idx = hash(key);
    size_t start_idx = idx;

    while (table_[idx].is_occupied) {
        if (table_[idx].key == key) {
            return table_[idx].value; // Found!
        }
        idx = (idx + 1) % TableSize;  // Probe forward
        if (idx == start_idx) break;  // Full loop safety check
    }
    return nullptr;                   // Not found
}
```

Store the start of the search

standard search loop to find the key.

```
void erase(OrderId key) {
    size_t idx = hash(key);
    size_t start_idx = idx;

    while (table_[idx].is_occupied) {
        if (table_[idx].key == key) {
            table_[idx].is_occupied = false;
            table_[idx].key = 0;
            table_[idx].value = nullptr;
            return;
        }
        idx = (idx + 1) % TableSize;
        if (idx == start_idx) break;
    }
}
```

Same logic as before except we set the occupied flag to FALSE so we can write over it, preserving the memory

```
bool add_order(OrderId id, Price price, Quantity qty, Side
side, PriceLevel& level) {
    Order* order = pool_.allocate();
    if (!order) return false;

    order->id = id;
    order->price = price;
    order->quantity = qty;
    order->side = side;

    id_to_order_.insert(id, order); // 1. Map ID to memory
pointer
    level.push_back(order);       // 2. Add to FIFO price
queue
    return true;
  }
```

Gets the pre-allocate node from the pool, populates it, registers it in the hash map.

```
bool cancel_order(OrderId id, PriceLevel& level) {
    Order* order = id_to_order_.find(id); // O(1) lookup
    if (!order) return false;

    level.remove(order);              // O(1) unlinking from
FIFO queue
    id_to_order_.erase(id);           // Clear from hash map
    pool_.deallocate(order);          // Return memory to
pool
    return true;
  }
};
```

finds the pointer in O(1) via the hash map, unlinks it from the price level in O(1) via pointers.